

Especificação Técnica v2: Sistema de BI Containerizado (Docker)

Para: Engenharia de Dados & DevOps **Objetivo:** Deployment do Sistema de BI em microserviços Docker para demonstração ao vivo. **Stack:** Docker Compose, PostgreSQL (DW), Streamlit (App), Python (ETL).

1. Arquitetura de Microserviços (Docker)

O sistema será composto por 3 contentores orquestrados via `docker-compose`.

Serviços:

- db_warehouse (PostgreSQL):**
 - Substitui o SQLite para permitir acesso via rede TCP/IP dentro da malha Docker.
 - Persistência de dados via Docker Volumes.
- app_dashboard (Streamlit):**
 - Exposto na porta `8501` para acesso Web.
 - Conecta-se ao `db_warehouse` para ler dados.
- etl_job (Python):**
 - Contentor efêmero (run-once) ou agendado.
 - Executa o pipeline: Ler CSV -> Inferência AI (pkl) -> Carga no Postgres.

Rede (Docker Network):

- Rede interna: `bi_network` (isolada).
- Apenas o `app_dashboard` expõe portas para o host (Demo).

2. Especificação do Data Warehouse (PostgreSQL)

Mudança de SQLite para Postgres para suportar containers.

Configuração:

- Imagem:** `postgres:15-alpine` (Leve e rápida).
- Variáveis de Ambiente:** `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`.

Schema (Adaptação para SQL): As tabelas mantêm a estrutura *Star Schema* definida anteriormente, mas com tipos de dados PostgreSQL (`SERIAL` em vez de `AUTOINCREMENT`, `TIMESTAMP`, etc.).

3. Especificação do Pipeline ETL (Container `etl_job`)

O script ETL deve ser robusto à conexão de rede (aguardar que a BD esteja pronta).

Fluxo de Execução:

1. **Wait-for-DB:** O script verifica se o serviço `db_warehouse` na porta `5432` está acessível.
2. **Schema Init:** Se as tabelas não existirem, executa o DDL (`CREATE TABLE...`).
3. **Extract & Transform:**
 - Lê `data/raw_input.csv` (montado via volume).
 - Aplica limpeza e *Feature Engineering*.
4. **AI Inference (Enrichment):**
 - Carrega modelos `.pkl` (montados via volume ou copiados no build).
 - Gera colunas `pred_stress` , `cluster` , etc.
5. **Load:** Insere os dados enriquecidos no PostgreSQL usando `sqlalchemy` ou `psycopg2` .

4. Especificação do Dashboard (Container `app_dashboard`)

Dockerfile:

- Base: `python:3.9-slim`
- Dependências: `streamlit` , `psycopg2-binary` , `pandas` , `scikit-learn` (para carregar o `scaler` se necessário na visualização), `matplotlib/seaborn` .
- Comando: `streamlit run app.py`

Interatividade:

- O dashboard conecta-se à BD usando o host `db_warehouse` (DNS interno do Docker).
- **Demo ao Vivo:** Acessível em `http://localhost:8501` .

5. Estrutura de Ficheiros (Atualizada para Docker)

```

/project-root
├── docker-compose.yml      # Orquestração dos 3 serviços
├── .env                   # Credenciais de BD (não comitar segredos reais)
├── /data
│   └── raw_input.csv       # Volume montado para o ETL ler
├── /models
│   ├── modelo_stress.pkl
│   └── ...
├── /database
│   └── init_script.sql     # (Opcional) DDL inicial se não feito via Python
├── /etl
│   ├── Dockerfile
│   ├── requirements.txt
│   └── etl_pipeline.py
└── /dashboard
    ├── Dockerfile
    └── requirements.txt
  
```

6. Plano de Implementação (Passo a Passo)

Passo 1: Docker Compose (A Espinha Dorsal)

Criar o `docker-compose.yml` definindo:

- Volume persistente para o Postgres (`pgdata`).
- Dependências (`depends_on`): Dashboard e ETL dependem da DB estar *healthy*.

Passo 2: Adaptação do Código Python

- **ETL:** Alterar a string de conexão de `sqlite:///mental_health.db` para `postgresql://user:pass@db_warehouse:5432/bi_db`.
- **Dashboard:** Alterar a conexão de leitura da mesma forma.

Passo 3: Dockerfiles

- Criar imagens otimizadas (camadas de cache para `pip install` para builds rápidos durante a demo).

Passo 4: Teste da Demo (O "Happy Path")

O Engenheiro deve garantir que este comando execute tudo:

```
docker-compose up --build
```

Resultado esperado na Demo:

1. Logs mostram a BD a iniciar.
2. Logs mostram o ETL a conectar, processar os modelos e inserir X linhas.
3. Logs mostram o Streamlit a ficar online.
4. Abrir o browser e mostrar os dados carregados.